MACARAEG, Alvin R. 6BLOCKTECH

Finals Lab Activity 1: Coding and TestingERC 721

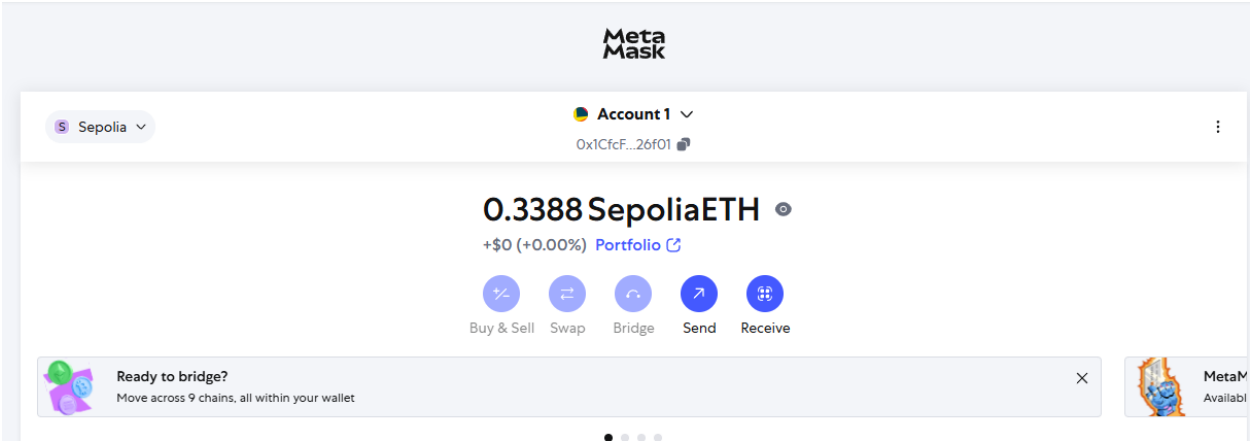
Part 1 — Setup: MetaMask Sepolia + Test ETH

Task 1.1: Enable Sepolia in MetaMask

1. Open MetaMask.
2. Go to **Settings** → **Advanced**.
3. Turn ON **Show test networks**.
4. Open the network dropdown and select **Sepolia**.

Checkpoint: MetaMask network shows **Sepolia**.

Sepolia has Value due to previous mining activities.



Task 1.2: Get Sepolia ETH from a Faucet

Use any faucet you prefer. Two common options:

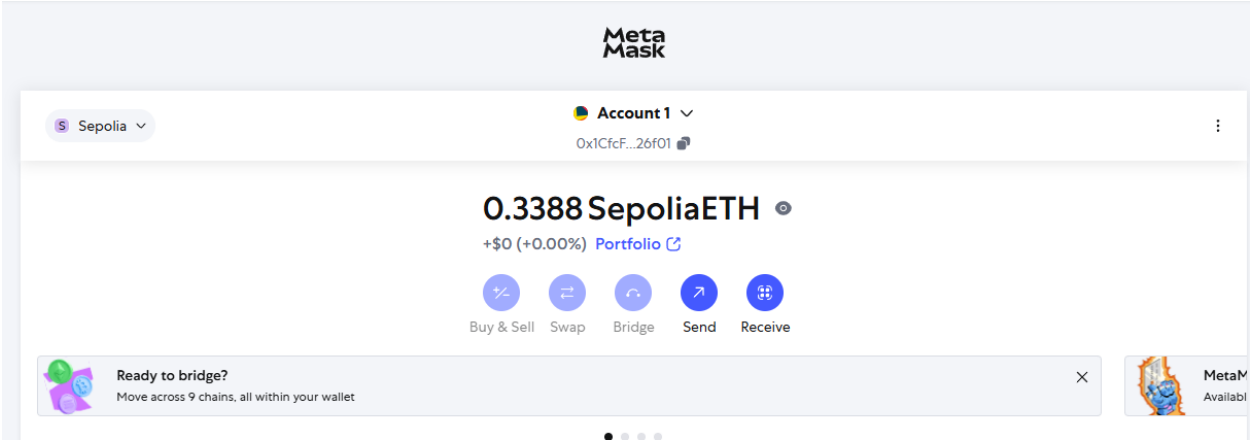
- MetaMask testnet faucet
- Chainlink Sepolia faucet

1. Copy your MetaMask wallet address.
2. Request test ETH from the faucet.
3. Wait until the balance updates.

Checkpoint: You have at least **0.2 Sepolia ETH** (enough for deploy + multiple mint tests).

Finals Lab Activity 1: Coding and TestingERC 721

I don't think I need to do faucet because I have enough for deploy + mint tests.

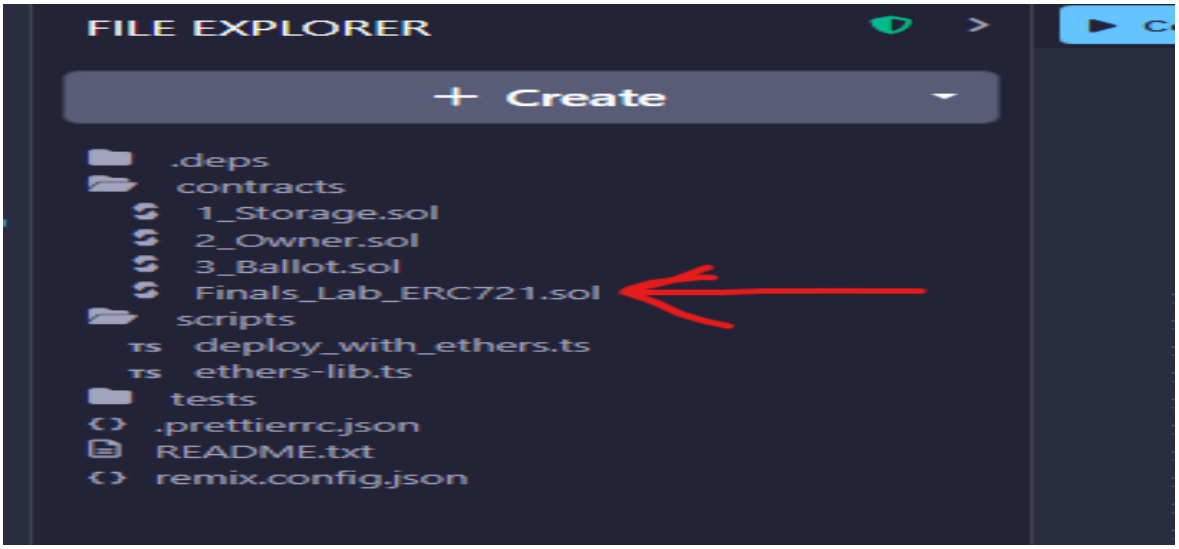


Part 2 — Remix Setup: Create the Contract File

Task 2.1: Open Remix and Create a File

1. Go to **remix.ethereum.org**
2. In **File Explorer**, click **New File**
3. Name it:
   Finals_Lab_ERC721.sol
4. Paste the contract code (exactly as given).

Checkpoint: File saved and visible in the explorer.



Finals Lab Activity 1: Coding and TestingERC 721

Part 3 — Dependencies: OpenZeppelin Imports

Task 3.1: Resolve OpenZeppelin Imports

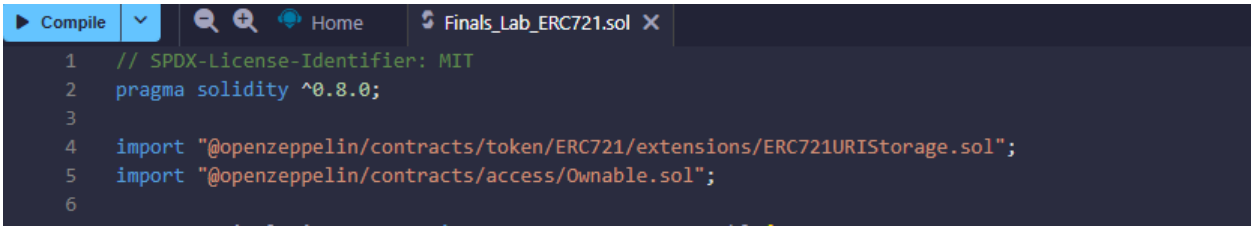
Your contract uses:

- Ownable.sol
- ERC721URIStorage.sol

1. Click **Solidity Compiler** tab.
2. Click **Compile** once.
3. If you see import errors (OpenZeppelin not found):
    - Go to **File Explorer**
    - Look for **Dependencies / Remix Packages**
    - Add/install **OpenZeppelin Contracts**

Checkpoint: The contract compiles without import errors.

No import errors. Now we can proceed.



Part 4 — Compile the Contract

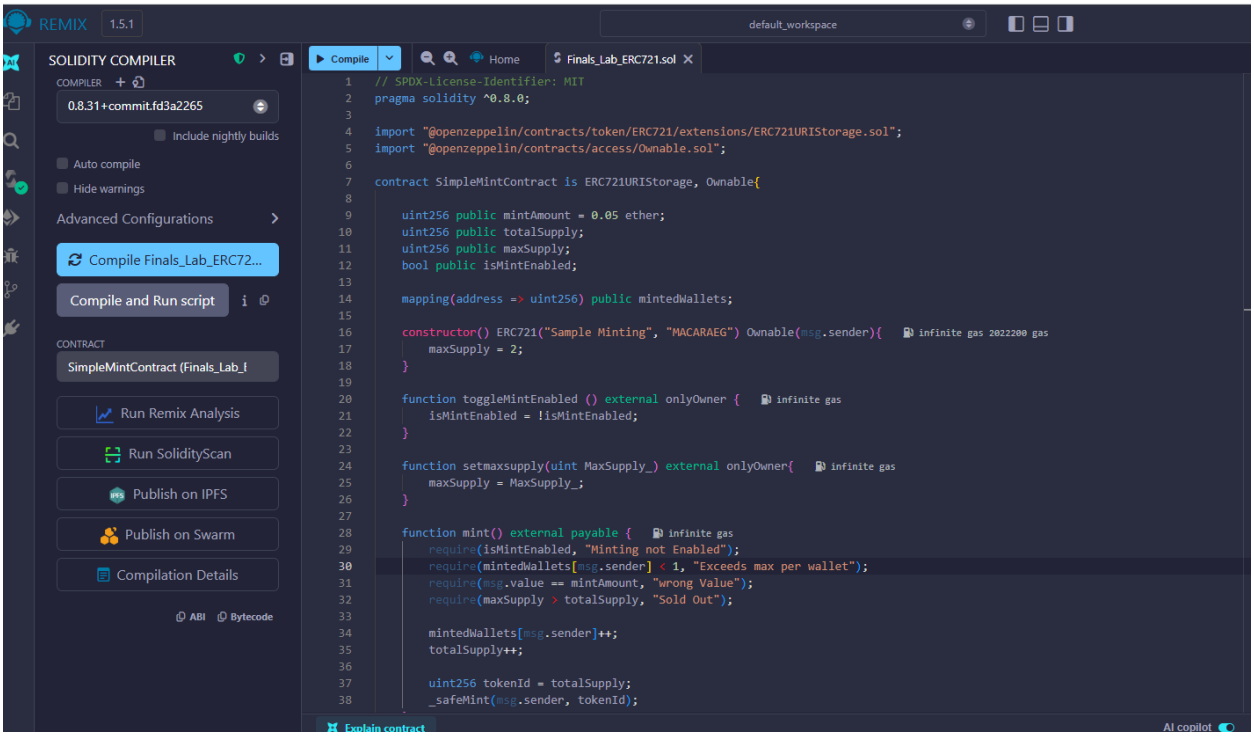
Task 4.1: Compile Settings

1. In **Solidity Compiler**:
    - Set compiler version to a **0.8.x** version (example: 0.8.20+)
    - Ensure it matches pragma solidity ^0.8.0;
2. Click: **Compile SimpleMintContract.sol**

Checkpoint: “Compilation successful” appears and no red errors show.

Finals Lab Activity 1: Coding and TestingERC 721

Whole code compiles successfully and no errors.



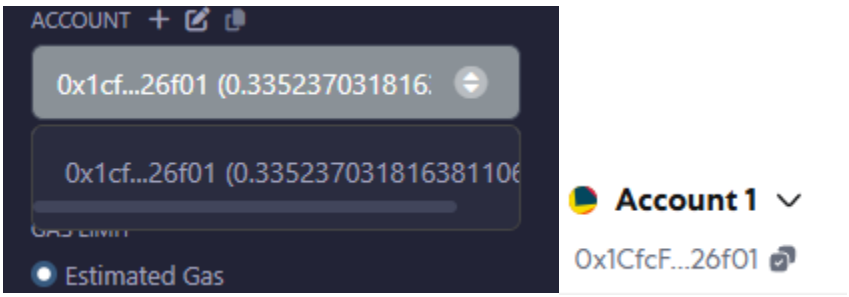
Part 5 — Connect Remix to MetaMask (Injected Provider)

Task 5.1: Use Injected Provider

1. Go to **Deploy & Run Transactions**
2. Under **Environment**, select:
    - **Injected Provider – MetaMask** (or "Injected Web3" depending on UI)
3. MetaMask will pop up:
    - Approve connection
4. Confirm MetaMask is still on **Sepolia**

Checkpoint: Remix shows your MetaMask address under "Account".

Finals Lab Activity 1: Coding and TestingERC 721



0x1CfcF9853Cc4255bd2C58c418466d2bb42426f01

End 26f01 corresponds.

Part 6 — Deploy the ERC-721 Contract

Task 6.1: Deploy

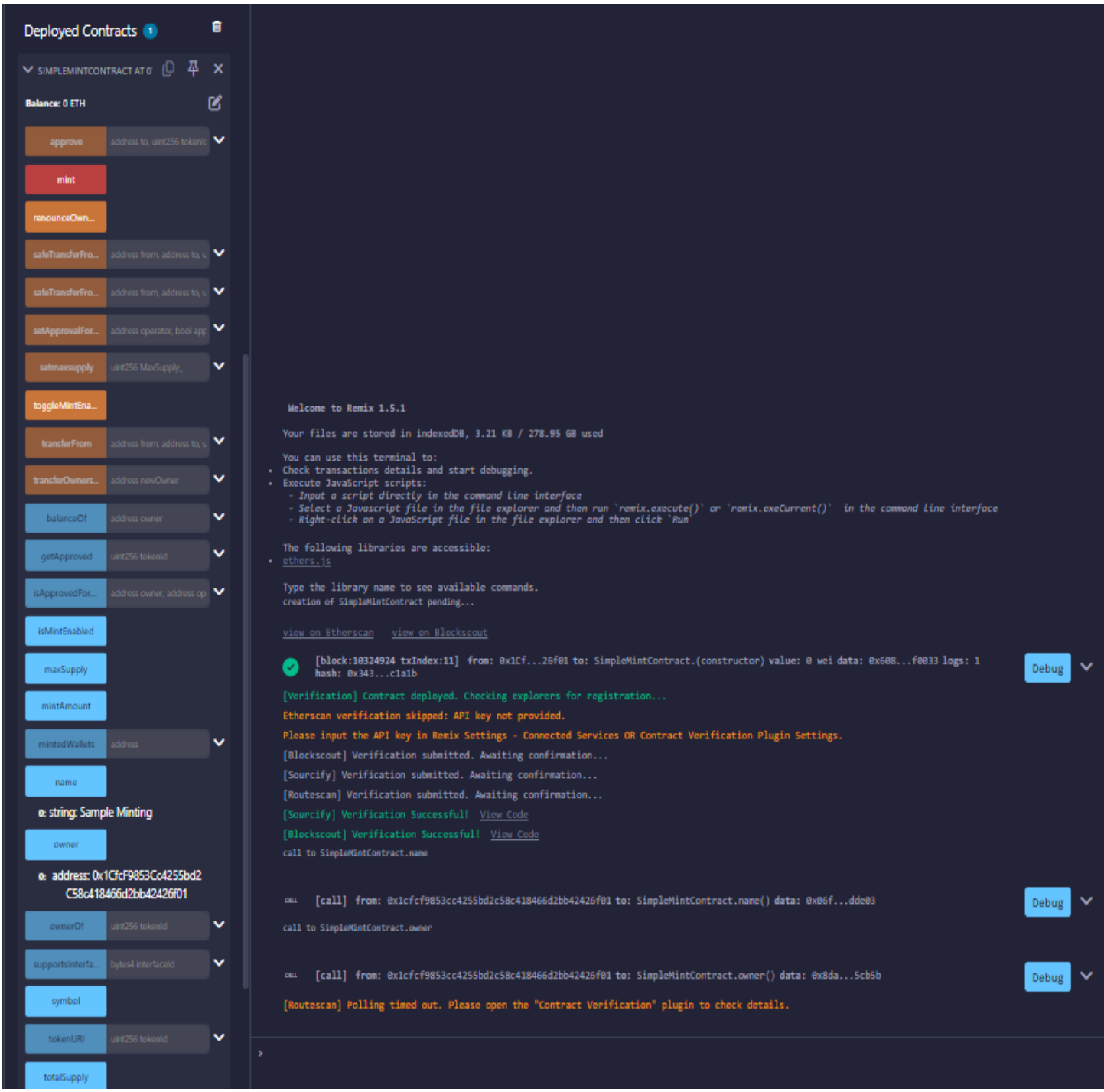
1. In Remix **Deploy** section:
    - Contract: Finals_Lab_ERC721.sol
2. Click **Deploy**
3. MetaMask pops up → Confirm

Checkpoint: Contract appears under **Deployed Contracts** in Remix

MACARAEG, Alvin R. 6BLOCKTECH

Finals Lab Activity 1: Coding and TestingERC 721

Deployed Contracts UI shows successful run.



Finals Lab Activity 1: Coding and TestingERC 721

Task 6.2: Validate Initial Contract State (Read Calls)

In **Deployed Contracts**, click these **read** functions:

Function	**Expected Value**
owner()	your MetaMask address
isMintEnabled()	false
maxSupply()	2
totalSupply()	0
mintAmount()	50000000000000000 wei (0.05 ETH)

Checkpoint: All values match.

Each call shows matching values.

owner() under 0: shows my account address.

```
from            0x1CfcF9853Cc4255bd2C58c418466d2bb42426f01

to              SimpleMintContract.owner() 0xc7378FBD93953572263Bb8D3B9c1B941A7874F61

input           0x8da...5cb5b

output          0000000000000282522491336019637912101971406513210221018766661111

decoded input   {}

decoded output  {
                    "0": "address: 0x1CfcF9853Cc4255bd2C58c418466d2bb42426f01"
                }

logs            []

raw logs        []
```

MACARAEG, Alvin R. 6BLOCKTECH

Finals Lab Activity 1: Coding and TestingERC 721

isMintEnabled(), shows false bool value.

```
CALL [call] from: 0x1cfcf9853cc4255bd2c58c418466d2bb42426f01 to: SimpleMintContract.isMintEnabled() data: 0x346...de50a

from            0x1CfcF9853Cc4255bd2C58c418466d2bb42426f01
to              SimpleMintContract.isMintEnabled() 0xc7378FBD93953572263Bb8D3B9c1B941A7874F61
input           0x346...de50a
output          0000000000000000000000000000000
decoded input   {}
decoded output  {
                    "0": "bool: false"
                }
logs            []
raw logs        []
```

maxSupply() , returns a value of 2.

```
CALL [call] from: 0x1cfcf9853cc4255bd2c58c418466d2bb42426f01 to: SimpleMintContract.maxSupply() dat

from            0x1CfcF9853Cc4255bd2C58c418466d2bb42426f01
to              SimpleMintContract.maxSupply() 0xc7378FBD93953572263Bb8D3B9c1B941A7874F61
input           0xd5a...beb01
output          0000000000000000000000000000002
decoded input   {}
decoded output  {
                    "0": "uint256: 2"
                }
logs            []
raw logs        []
```

totalSuplly() , shows a value of 0

```
CALL [call] from: 0x1cfcf9853cc4255bd2c58c418466d2bb42426f01 to: SimpleMintContract.totalSupply() data: 0x181...60ddd

from            0x1CfcF9853Cc4255bd2C58c418466d2bb42426f01
to              SimpleMintContract.totalSupply() 0xc7378FBD93953572263Bb8D3B9c1B941A7874F61
input           0x181...60ddd
output          0000000000000000000000000000000
decoded input   {}
decoded output  {
                    "0": "uint256: 0"
                }
logs            []
raw logs        []
```

Finals Lab Activity 1: Coding and TestingERC 721

mintAmount(), returns uint value 50000000000000000 wei (0.05 ETH)

Part 7 — Functional Test Cases (Minting Rules)

Test Case 1 — Mint while Disabled (Should Fail)

Task 7.1: Attempt Mint (Expect Revert)

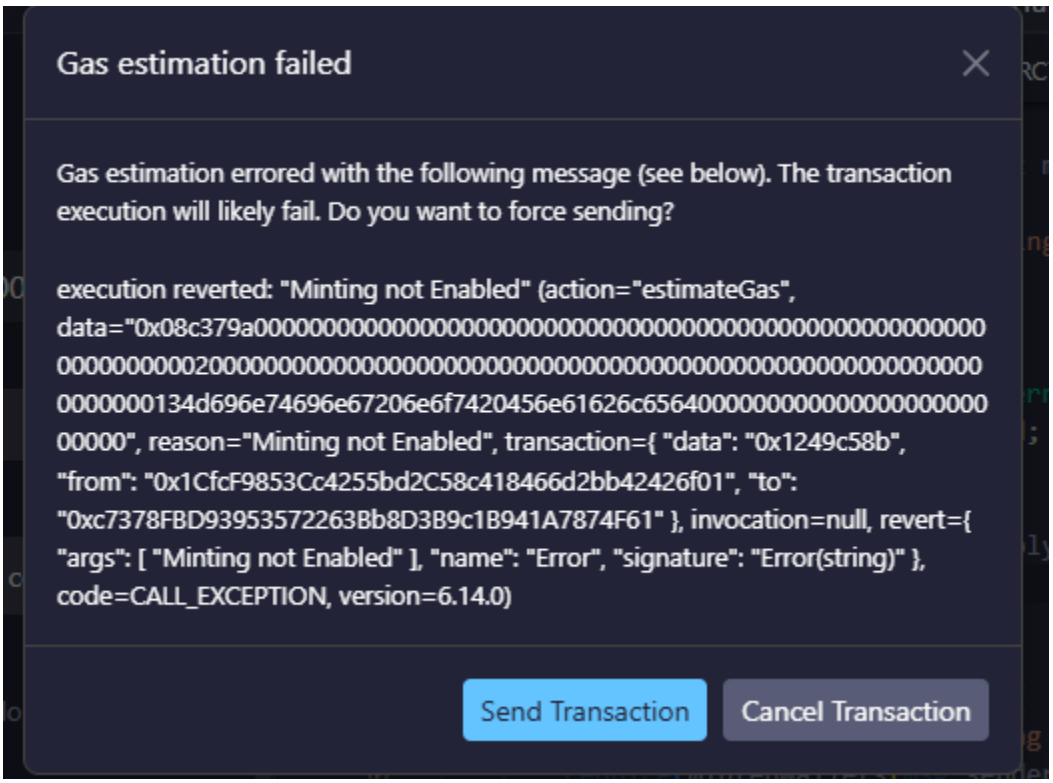
1. Expand mint()
2. In Remix top bar, set:
   - **Value:** 0.05
   - **Unit:** ether
3. Click **mint()**
4. Confirm transaction if MetaMask asks (it may fail before prompting depending on UI)

Expected Result: Transaction fails with:

- **"Minting not enabled"**

Record: Write the revert message in your lab notes.

Finals Lab Activity 1: Coding and TestingERC 721



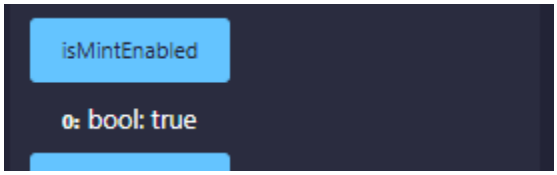
Test Case 2 — Enable Minting (Owner Only)

Task 7.2: Toggle Minting

1. Click toggleisMintEnabled()
2. Confirm in MetaMask
3. Call isMintEnabled() again

Expected Result:

- isMintEnabled() → true



Test Case 3 — Successful Mint (Correct Payment)

Task 7.3: Mint First NFT

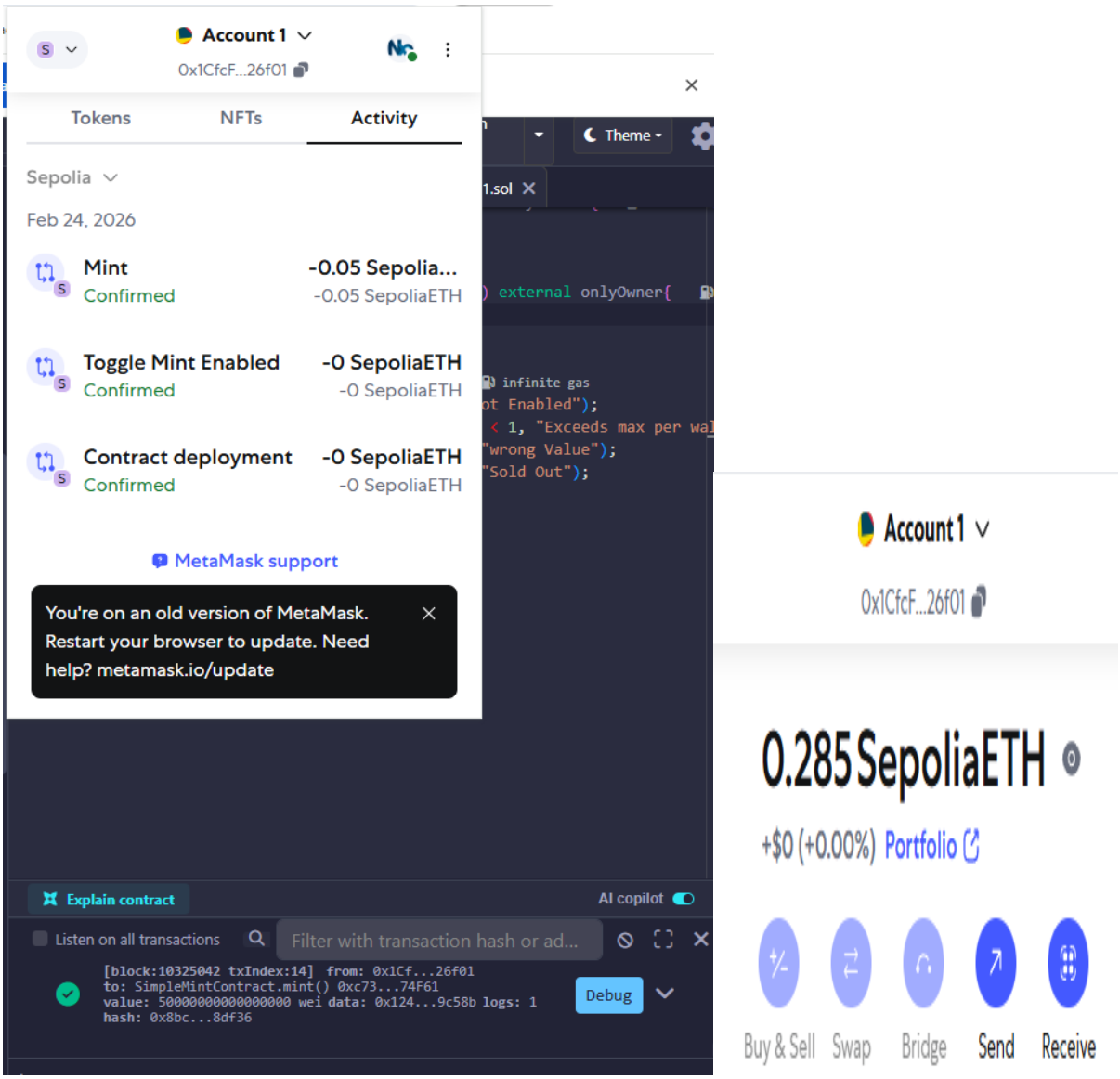
Finals Lab Activity 1: Coding and TestingERC 721

1. Set **Value = 0.05 ether**
2. Click mint()
3. Confirm MetaMask

Expected Result (after mined):

- totalSupply() → 1
- mintedWallets(yourAddress) → 1

Record: Transaction hash + updated values.



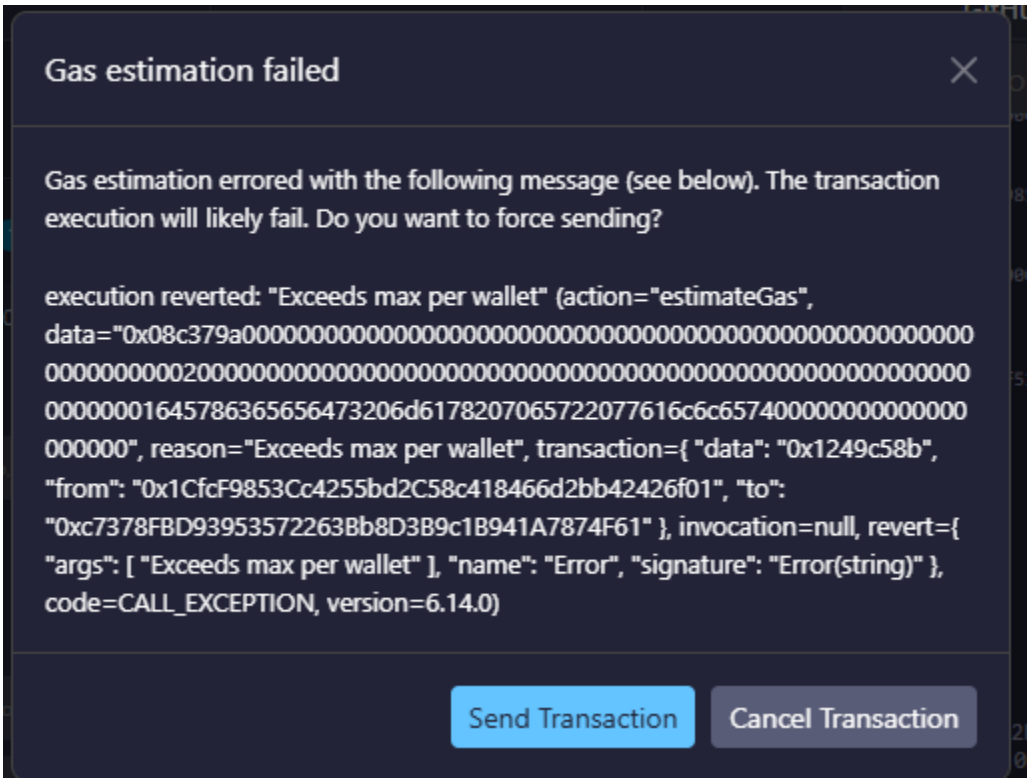
Finals Lab Activity 1: Coding and TestingERC 721

Test Case 4 — Mint Again Same Wallet (Should Fail)

Task 7.4: Attempt 2nd Mint from Same Wallet

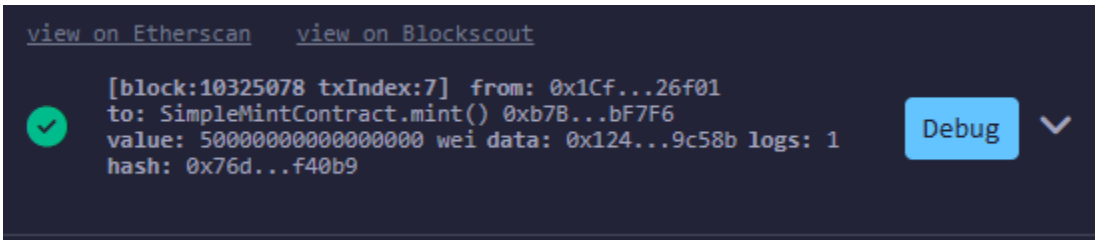
1. Set **Value = 0.05 ether**
2. Click mint()

Expected Result: Revert message: **"Exceeds max per wallet"**


Gas estimation failed
Gas estimation errored with the following message (see below). The transaction execution will likely fail. Do you want to force sending?
execution reverted: "Exceeds max per wallet" (action="estimateGas", data="0x08c379a0000000000000000000000000000000000000000000000000000000000000000000000000200000000000000000000000000000000000000000000000000000000000

4. Call mint() with **0.05 ether**

Expected Result: totalSupply() → 2



Test Case 6 — Sold Out (Should Fail)

Task 7.6: Attempt Another Mint (Any Wallet)

Try minting again with any account.

Expected Result: Revert message: **"Sold out"**

Part 8 — Common Gotchas (Troubleshooting Checklist)

Issue A: Wrong Network

Symptom: Transactions fail / contract not visible / wrong chain

- Fix: Ensure MetaMask network is **Sepolia**
- Reminder: Injected Provider follows MetaMask network

Issue B: Wrong Value Sent

Symptom: Revert “Wrong value”

- Fix: Your contract requires:
    - msg.value == mintAmount exactly
- Use **0.05 ether** or **50000000000000000 wei**

Issue C: ERC721URIStorage “Unused”

Observation: You imported URI storage but did not set token URIs.

- Result: You can mint NFTs, but **they won’t have metadata URIs** unless you implement _setTokenURI.

Finals Lab Activity 1: Coding and TestingERC 721

SOLD OUT

Part 9 — Post-Lab Questions (Assessment)

Answer briefly:

1. Why did mint() fail before enabling minting?
   - The function triggered the first require check: require(isMintEnabled, "Minting not Enabled");. Since the boolean isMintEnabled defaults to false when deployed, the contract will revert any minting attempts until the owner calls toggleMintEnabled().

2. What specific line enforces “1 mint per wallet”?
   - require(mintedWallets[msg.sender] < 1, "Exceeds max per wallet");.
3. What happens if you send 0.051 ETH?
   - The transaction will fail/revert with the error message "wrong Value". This is because the contract performs a strict equality check (msg.value == mintAmount) and 0.051 does not exactly match the required 0.05.

4. Why do you need a second wallet to reach maxSupply = 2?
   - Because of the restriction mentioned above (mintedWallets[msg.sender] < 1), a wallet address can only successfully call the mint() function once. In order to reach the total maxSupply of 2, a different wallet address is required for the second mint.
5. What is the value of mintAmount in **wei**?

   =- 50,000,000,000,000,000 wei (which is 0.05×10^{18} wei).

Part 10 — Optional Extension (Metadata Upgrade Challenge)

Challenge Goal

Upgrade the contract so each minted token has a metadata URI.

Suggested approach

- Modify mint() to accept a URI:

- mint(string memory uri) external payable

- After _safeMint, call:
  - _setTokenURI(tokenId, uri);

Success criteria:

- After minting, calling tokenURI(tokenId) returns the URI you provided.

Student Output Checklist (What to Submit)

Screenshots of the following:

1.
    1. MetaMask showing Sepolia network + balance
    2. Remix deployed contract
    3. Successful mint transaction (1st wallet)
    4. Revert: “Minting not enabled”
    5. Revert: “Exceeds max per wallet”
    6. Successful mint transaction (2nd wallet)
    7. Revert: “Sold out”